

[Project Name]
Documentation

[Team Number]
[Member A]
[Member B]
[Member C]
[Member D]
[Member E]

[Date]

Contents

Change Log

Watch out: Treating early iterations as "draft"

Do not use placeholder text or "to be decided" sections anywhere in this document. Each iteration should be written as a complete, reviewable deliverable – assume it is final unless a later iteration explicitly revises it.

Iteration 1:

- *[What was created/changed in Iteration 1]*

Iteration 2:

- *[What was created/changed in Iteration 2]*

Iteration 3:

- *[What was created/changed in Iteration 3]*

Final Iteration:

- *[What was created/changed in the Final Iteration]*

1. Requirements Analysis

Watch out: Analysis before implementation

Finish this section and the Use Cases section *before* you discuss any classes, code, or UI screens elsewhere in the document. Every planned feature must trace to at least one requirement and one use case. Avoid phrases like "we plan to implement ..." without first explaining *why* the feature is needed from the user's perspective.

Customer / Stakeholder Perspective: *[Describe who the end users are and what they are trying to achieve, in their own words.]*

Functional Requirements:

- *[Functional requirement 1]*
- *[Functional requirement 2]*
- *[Functional requirement 3]*

Non-Functional Requirements / Environment:

- *[Platform, performance, deployment constraints, etc.]*

2. User Story

Watch out: Requirements traceability

Every requirement above should be reflected in at least one user story, and every user story should later map to a use case. Keep IDs or consistent wording so a reader can trace a requirement all the way through to the System Sequence Diagrams.

- As a *[role]*, I want to *[goal]* so that *[benefit]*.
- As a *[role]*, I want to *[goal]* so that *[benefit]*.
- As a *[role]*, I want to *[goal]* so that *[benefit]*.

3. Key Technical Challenges

- *[Challenge 1 and how it will be approached]*
- *[Challenge 2 and how it will be approached]*

Key Risks

- *[Risk 1]*
- *[Risk 2]*

Competition

- *[Competitor / existing solution 1 – what it does, what it lacks]*
- *[Competitor / existing solution 2]*

4. What's New?

[Summarize what is novel about this project compared to existing solutions listed under Competition.]

5. Workflow

Watch out: System boundary

Make sure the workflow diagram clearly separates what the *actor* does from what the *system* does. Actors should never be shown performing system responsibilities (e.g., "system authenticates," not "user authenticates"), and external systems should not be drawn as if they were internal components.

[Insert workflow / activity diagram here, e.g.
`\includegraphics[width=\textwidth]{workflow.png}`*]*

6. Use Cases

Watch out: Use cases are not UI click-throughs

Do not write use cases as a sequence of UI actions (“user clicks button, system shows page”). Describe the *system’s responsibilities and guarantees* – business rules, alternative flows, and constraints – not screen navigation. UI details will change over time; system behavior should be the stable part of the documentation.

6.1 *[Subsystem / Module Name (e.g., Client Application)]*

Actor: *[Actor name]*

Use Cases: *[Use case 1], [Use case 2], [Use case 3]*

Actor Description: *[One or two sentences describing the actor.]*

Use Case: *[Name]*

[Short description of the use case.]

Actor: <i>[Actor]</i>	System: <i>[System Name]</i>
1. <i>[Actor step]</i>	
	2. <i>[System step]</i>
3. <i>[Actor step]</i>	

Repeat the subsection above for each use case, and repeat the section above for each subsystem/actor group (e.g., a Server / Manager-facing module, following the pattern of the Client module).

7. Use Case Diagrams

[Insert use case diagram(s) here, one per subsystem, e.g.
\includegraphics[width=0.8\textwidth]{use-case-diagram.png}]

8. System Sequence Diagram

Watch out: Consistency with use cases

Every scenario shown here should correspond to a use case described earlier, using the same name. Missing or renamed scenarios break traceability between Use Cases and SSDs.

[Scenario Name]

[Insert system sequence diagram here.]

Actor	System
1. <i>[step]</i>	
	2. * <i>[system response]</i>
3. <i>[step]</i>	

Repeat for each scenario.

9. Domain Model

Watch out: Domain model is not a class diagram

Use only nouns from the problem statement as domain classes. Do not include methods, data types, UI classes, or persistence/database details – those belong to design, not analysis. Show entities and the associations/multiplicities between them, not implementation.

*[Insert domain model diagram here, e.g.
`\includegraphics[width=\textwidth]{domain_model.png}`]*

10. Test Cases

Watch out: Traceability of tests

Each test case should reference the requirement or use case it verifies. Keep test case IDs consistent across iterations so gaps in coverage are easy to spot.

10.1 Test Cases for Iteration 1

Target Users: *[Who]*

What is covered in the tests:

1. *[Area of functionality covered]*

Test Case: *[ID, e.g., T1]*

- **Description:** *[What this test checks]*
- **Action:** *[Function/action exercised]*
- **Related Requirements:** *[Preconditions]*
- **Expected Result:** *[Expected behavior]*
- **Test Result:** *[Actual behavior observed]*
- **Status:** *[Pass / Fail]*

Repeat the Test Case block above for each test in this iteration.

10.2 Test Cases for Iteration 2

[Same structure as Iteration 1.]

10.3 Test Cases for Iteration 3

[Same structure as Iteration 1.]

10.4 Test Cases for Final Iteration

[Same structure as Iteration 1.]